
Hardware and Systems Strategies for Neural Acceleration

Shelby Thomas

Computer Science and Engineering
University of California, San Diego
San Diego, CA 92122
sht005@eng.ucsd.edu

Abstract

For years the development of new neural networks and better training algorithms has been largely orthogonal to what kind of hardware might drive their usage. While neural networks have been around since the 1980s they were considered far too computationally intensive and impractical compared to canonical computer vision techniques. In 2012 a neural net named AlexNet handily broke the world record for image recognition tasks using a deep convolution neural network trained on commodity GPUs. This showed for the first time that deep convolutional neural networks can be trained on a low cost system in a reasonable time, one week, while producing state-of-the-art accuracy. Since then the exponential growth of even deeper neural network research has been enabled through high performance CPU/GPU devices and an awareness of the underlying hardware. This has opened the door for an emerging class of neural systems that requires an understanding of the neural learning problem coupled with hardware expertise. This survey presents an overview of the bottlenecks in neural network algorithms and the hardware and software systems designed to overcome them. A new taxonomy for research in neural acceleration is also presented that represent the three angles most commonly taken to optimize this algorithm - technology enabled acceleration (TEA), system enabled acceleration (SEA) and algorithm enabled acceleration (AEA).

1 Introduction

A neural network is a machine learning technique to generate a generalizable model for a certain problem domain. Broadly, machine learning techniques fall into two categories: *Supervised learning* and *unsupervised learning*. Supervised learning involves learning from training examples and target outputs while unsupervised learning involves extracting features from data without positive and negative examples. At a high level the goal of any machine learning algorithm is to generate a function that can model a certain phenomenon. This phenomenon can be the relationship between house features and house price, or pixels and output image. The model itself is completely dependent on training data but needs to generalize enough to perform well on data that it has never seen before. It is helpful to imagine machine learning as a function approximator, the goal of training is to create a function with the correct set of constants or weights that minimize the error on the output. For most machine learning techniques this involves solving a convex optimization problem and finding a global minimum. In contrast, neural networks generate and model nonlinear functions and require solving a significantly more involved non-convex optimization problem with several local minima and no guarantee that the global minima will be achieved. While there are many variants of neural networks, they are built on top of a set of common characteristics and techniques. This basic architecture can be seen in Figure 1 - the input layer has four neurons and the output layer has two neurons. The layers in the middle with five and three neurons make up the hidden layers of the network. Inputs of the network are propagated from the input layer to the hidden layers and produce a prediction

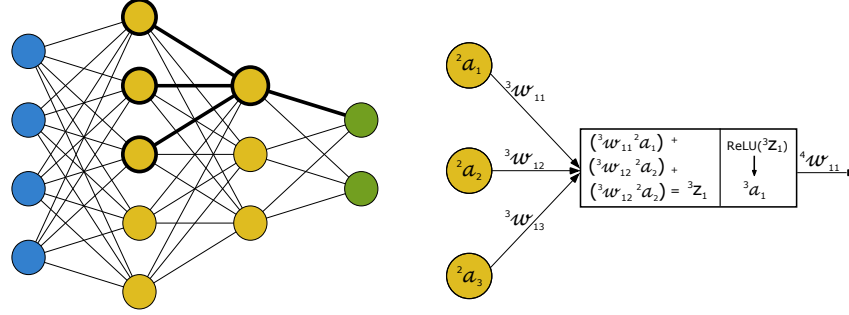


Figure 1: Multi-layer perceptron: Computational operations performed in a neural network.

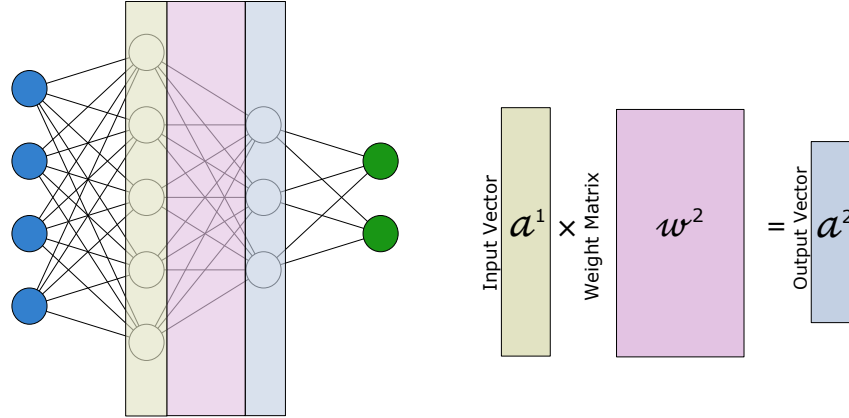


Figure 2: The dense vector-matrix multiply operation common in fully connected networks.

in the output layer. By increasing the number of neurons or the number of layers in the hidden layers, the network is able to approximate more fine grain features but is prone to overfitting and poor generalization.

2 Neural Network Overview

A typical net consists of a number of layers $l \dots L$ defined by a vector of activation units (neurons) and matrix of edges weights (synapses) between neurons. In this section we will discuss the most basic type of neural network, a multi-layer perceptron (MLP) shown in Figure 1 (left). In this kind of network all of the nodes are *fully connected* meaning that every neuron in a layer has an edge to every other neuron in the next layer. Note that in this fully connected architecture the number of edges between two layers is the product of the neurons in each layer, i.e. four neurons in layer one and five neurons in layer two will have 20 edge weights.

2.1 Computational Rules

We can distinguish a unique neuron a_j^l by index j and the layer l it is associated with i.e. neuron a_3^2 is the *third neuron in layer two*. Generally it is helpful to think of a group of neurons in one layer in terms of vectors in which case we omit the subscript and keep the layer superscript i.e. a^2 is the *vector of activations in layer two*.

Unique edges are defined similarly as W_{ij}^l where the index of an edge denotes a "to-from" relationship - to the i neuron in layer $l + 1$ from the j neuron in layer l , an example of this is shown in Figure 1 (right). Similar to the vector of neurons it is useful to represent all the edges between neurons as a matrix of weights denoted with a layer superscript i.e. W^2 is the *weight matrix of the second layer that contains edges from layer two to layer one*.

The fundamental unit of computation in a neural network is a dense vector-matrix multiplication between the input activations vector a^l and the weight matrix W^{l+1} . The vector-matrix multiplication is a set of multiply-accumulate operations that provide a new vector z^{l+1} , also known as the *weighted input vector*. The weighted inputs are passed through an *activation function* g which introduces a non-linearity into the network and produces the output vector a^{l+1} . [-] In Figure 1 (right) we show explicitly each of the aforementioned computations in terms of this notational structure. We choose to use this standard notation as it simplifies our discussion during the training (backward-pass) phase.

Putting all of this together, the output of our computation can be defined as an equation as a function of the input layer a^0 and W where $W = \{W^l\}$. In the 3-layer MLP case this equation is defined as:

$$f(a^0; W) = W^3(g(W^2g(W^1a^0))) \quad (1)$$

3 The Learning Problem

Note that the goal of our learning algorithm is to generate a function that models our data. This is achieved by tuning the weights W of our network such that we reduce error between the collected results or *ground truth* and the results that the network outputs. We quantify how well a network performs by using a *loss or objective function* $\mathcal{L}$ and tuning our weights W such that error is reduced. We can pose this as the following optimization problem:

$$\min_W \mathcal{L}(f(a^0; W), y) \quad (2)$$

Once the weights have converged, the network has completed training and the weights will not change again unless retrained. This means that weights can be trained on an external machine and then exported to an embedded device that uses the same network architecture to perform *inference*. Training time takes significantly longer than inference as solving the non-convex optimization problem in Equation 2 requires several tens of thousands of iterations of an optimization technique such as *stochastic gradient descent* before the network converges. In contrast, making a prediction only requires us to solve Equation 1 using basic vector-matrix multiplies and take an order of magnitude less memory. Further explanation into the difference in computation and memory usage of inference and training is the topic of another section.

Formulating the Training Algorithm

Neural network training has three steps, the forward pass, backward pass, and weight updates. The forward evaluation runs through the network to the output and evaluates the error in the output layer relative to what the true output of the data should be. For clarity the layers are explicitly stated next to each weight w and activation a . The activation for a single neuron is the following:

$$a_i^l = g\left(\sum_{j=1}^k W_{ij}^l \times a_j^{l-1}\right) \quad (3)$$

The above equation states that the activation of a certain neuron a_i in layer l , is the product of every weight **to** neuron i **from** neuron j in layer $l - 1$ with the activation of the previous layer $l - 1$ of neuron a_j . This process is repeated for every neuron in $l - 1$ layer until neuron k and summed. The activation function $g(x)$ is then applied to the sum. Note that we have omitted an extra factor b_i called the bias neuron that allows a shift of the activation function. Computationally, it is one extra neuron in each layer added to the total and for clarity it is omitted.

Error terms δ are computed for each neuron i in the output layer l_o . Where $t(x)$ indicates the true label of the data and $g'(x)$ is the derivative of $g(x)$:

$$\delta_i^{l_o} = (t_i^{l_o} - a_i^{l_o}) \times g'(a_i^{l_o}) \quad (4)$$

To measure how much error each layer contributes to the total an error matrix is created that has the same dimensions as the weights between two layers. By taking partial derivatives of each layer with respect to its output layer, this recursive provides a measurement of how much each weight affects the final output. In a naive implementation, storing this error matrix essentially doubles the

memory requirement of network as we now need to store memory for the weights and the errors. Equation 5 is a general equation of calculating the error matrix for every layer:

$$\delta_i^l = \left(\sum_{j=1}^m \delta_j^{l+1} \times W_{ji}^l \right) \times g'(a_i^l) \quad (5)$$

Note that the Equation 4 is simply a special case of Equation 5. In Equation 4 we are calculating the derivatives with respect to the final output where in Equation 5 we are calculating derivatives with respect to the previous layer.

After the error matrices for each layer are calculated and stored, a small change in the weight matrix of each layer is made based on the error matrices and a pre-defined learning rate α . Equation 6 shows that for every layer l the change in the weight ΔW from neuron i to j is a product of the learning rate α , the error weight associated with that neuron, δ_i^l , and the old value of the neuron a . Note that we are performing almost the same calculation we do in the forward pass except our weight matrix is now the error matrix on which the learning rate is applied and we are applying changes on the weights backward through the network (from output layer to input layer). This entire training process is appropriately named **back-propagation**.

$$\Delta W_{ij}^l = \alpha \times \delta_i^l \times a_j^{l-1} \quad (6)$$

This weight update step of back-propagation is also known as gradient descent (GD) and is the optimization algorithm that we apply to the network. In practice an algorithm called stochastic gradient descent (SGD) with mini-batching is used. In GD one update is made after running through the entire dataset. For large datasets like ImageNet with millions of data points, performing one update per million samples makes training almost impossible. SGD takes the opposite route where parameter updates are made after only one sample. The problem with this method is that since every example has an equal effect on the network, outliers and noisy points have the potential to swing the network away from the optima. In practice a middle ground between the two methods is employed using mini-batches of data to make a single update. This ensures faster training speed than GD and averages out the noisy points of the single example SGD method. **The trade-off for this optimization is a significantly higher memory footprint as our storage requirements increase by (approximately) a factor of the mini-batch size.**

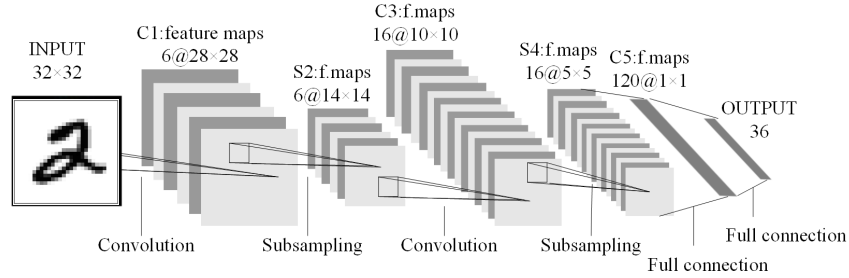


Figure 3: The LeNet-5 architecture

4 Convolutional Neural Networks

Convolutional neural networks (CNN) have shown state-of-the-art performance in a variety of image, speech, and video recognition tasks beating long standing canonical methods of computer vision and language processing. Unlike the densely connected MLPs the weights or filters in a CNN are locally connected and the activations make up the output images. While this local connectivity means that there are fewer weights in each layer compared to the MLP, the total memory usage of the activations is far greater. Figure 3 shows an example of a CNN. In this image the filters are omitted and just the activation feature maps are shown. After every convolution layer the image size is slightly reduced as a function of the convolution operation but in some layers an explicit sub-sampling process is performed to aggregate multiple low-level features.

4.1 Understanding the Convolution Operation

The primary computational kernel in a CNN is a convolution operation between an input image f and a *filter or kernel* g . Figure 4 shows the resulting feature map after convolving an edge detection filter.

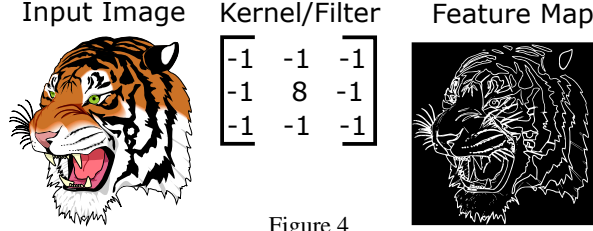


Figure 4

In a continuous one dimensional domain, convolving two signals $f * g$ is equivalent to taking the integral of the pointwise multiplication of these two function as a function of the amount that one of the signals g is translated. This is summarized in Equation 7.

$$h(x) = f * g = \int_{-\infty}^{\infty} f(x - u)g(u)du \quad (7)$$

When working with images the we use a discrete 2D domain (pixel level rows and columns) formula. Note that the convolution theorem states that *under suitable conditions the Fourier transform of a convolution is the pointwise product of Fourier transforms*. Equation 8 shows the discrete convolution theorem for image data. In the first method, the spatial domain, a filter of size x, y is passed through the input image of size (a, b) as we iterate through the rows and columns of the image matrix to produce an output feature map. Alternatively, if we perform this operation in the Fourier domain we can significantly reduce the amount of operations but at the cost of more memory (storing intermediate FFT matrices). Note that $\sqrt{2\pi}$ here is a normalization factor.

$$\begin{aligned} \text{feature map}(a, b) &= \text{input} * \text{kernel} = \sum_{y=0}^{\text{columns}} \sum_{x=0}^{\text{rows}} \text{input}(a - x, b - y) \text{kernel}(x, y) \\ &= \mathcal{F}^{-1} \left(\sqrt{2\pi} \mathcal{F}[f] \mathcal{F}[g] \right) \end{aligned} \quad (8)$$

4.2 Algorithm Overview

In contrast to MLPs, the neurons in a CNN are arranged in three dimensional *volumes*. For image recognition, the input of a CNN is a three dimensional image object of size $H \times W \times D$, for height, width, and dimensions respectively. The algorithm learns a set of filters K which are *convolved or slide* across the image to produce an output volume. The size of the filter is called the *receptive field* F and the *stride* S that the filter takes across the image determines the size of the output volume. Note that the filter has size $F \times F$ as filters are generally cubic volumes. To perform the final classification and generate a likelihood the final layer of a CNN is a MLP.

Similar to the MLP, the goal of learning in a CNN is to learn filters(weights) that reduce error on the output by learning different features about a dataset(images). For example, filters may learn to become rotation, scale, and translation invariant and learn different textures in an image. The total number of weights in a CNNs can be summarized as:

$$\sum_{i=0}^{\text{Conv. Layers}} (F_i)^2 \times K_i + \sum_{j=0}^{\text{Clas. Matrix}} \dot{W}^j \quad (9)$$

The left half of the equation is the sum of all the 3D filters in each convolution layer i with dimension $F \times F \times K$. The right half is the classification layers starting from layer j and is a sum over all classification weights $\dot{W}$ from j to the last classification layer.

Figure 5 shows an example of two filters convolving an input image of size $6 \times 6 \times 3$. The properties of this filter are $K = 2, F = 2, S = 1$. Each output filter map has size 5×5 and the total volume including the two filters is $5 \times 5 \times 2$.

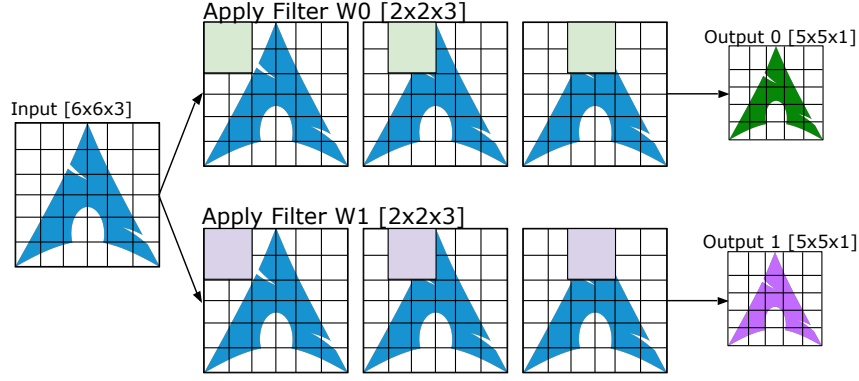


Figure 5: An example of a convolutional layer. This operation is repeated several times with varying filter sizes in every layer. In a CNN the weights of the network are the filters and the activations are the output feature maps.

5 Memory and Computation

There is a vast body of work involved in developing neural systems that can support next generation neural networks. One of the big driving factors for better neural hardware is the observation that the depth of a network correlates with higher accuracy. The set of networks from the VGG family of benchmarks[1] show that deeper networks produced high accuracy when experimenting from 16-19 layers on the ImageNet 1k dataset. Google's winning entry into the Large Scale Visual Recognition Challenge (ILSVRC) in 2014, GoogLeNet[2], has a 22-layer CNN based Inception Network architecture [3]. In 2015, a 30-layer neural network was proposed that produced even higher accuracy on the same ImageNet dataset[4]. Finally, in late 2015 Microsoft edged out the competition with a 152 layer CNN based network[5] in ILSVRC-2015. The first time a CNN won the international image recognition challenge ILSVRC was in 2012 with AlexNet[6] which was 9 layer network with an error rate of 16%. In 2015 competition, Microsoft's Deep Residual Network achieved an error rate of 3.57% surpassing the rate of human accuracy.

INPUT: [224x224x3]	memory: 224*224*3=150K	weights: 0
CONV3-64: [224x224x64]	memory: 224*224*64=3.2M	weights: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64]	memory: 224*224*64=3.2M	weights: (3*3*64)*64 = 36,864
POOL2: [112x112x64]	memory: 112*112*64=800K	weights: 0
CONV3-128: [112x112x128]	memory: 112*112*128=1.6M	weights: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128]	memory: 112*112*128=1.6M	weights: (3*3*128)*128 = 147,456
POOL2: [56x56x128]	memory: 56*56*128=400K	weights: 0
CONV3-256: [56x56x256]	memory: 56*56*256=800K	weights: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256]	memory: 56*56*256=800K	weights: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256]	memory: 56*56*256=800K	weights: (3*3*256)*256 = 589,824
POOL2: [28x28x256]	memory: 28*28*256=200K	weights: 0
CONV3-512: [28x28x512]	memory: 28*28*512=400K	weights: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512]	memory: 28*28*512=400K	weights: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512]	memory: 28*28*512=400K	weights: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512]	memory: 14*14*512=100K	weights: 0
CONV3-512: [14x14x512]	memory: 14*14*512=100K	weights: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512]	memory: 14*14*512=100K	weights: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512]	memory: 14*14*512=100K	weights: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512]	memory: 7*7*512=25K	weights: 0
FC: [1x1x4096]	memory: 4096	weights: 7*7*512*4096 = 102,760,448
FC: [1x1x4096]	memory: 4096	weights: 4096*4096 = 16,777,216
FC: [1x1x1000]	memory: 1000	weights: 4096*1000 = 4,096,000
Training:		
Static Memory	: 138M parameters * 4 bytes ~ 0.5GB	
(Forward) Dynamic memory	: 15M * 4 bytes * 256 batch ~ 15GB	
(Backward) Dynamic memory	: 15M * 4 bytes * 256 batch * 2 ~ 30GB	
Inference:		
Static Memory	: 138M parameters * 4 bytes ~ 0.5GB	
(Single Image) Dynamic memory	: 15M * 4 bytes ~ 93MB/image	

Figure 6: Memory usage for VGGNet. The network was trained on four Nvidia Titan Black GPUs and took three weeks to converge.

5.1 Neural Network Memory Requirements

The memory requirements of a neural network are broken into two major parts, static memory and dynamic memory usage. **Static memory** refers to the total amount of weights or parameters in the network multiplied by 4, the number of bytes per weight. The static memory of the network define the entire architecture and these parameters are small enough to be exported onto an embedded system while using less than half a gigabyte of memory for state of the art models. **Dynamic memory** is the memory usage for the activations that need to be stored during intermediate steps of inference or training. During training, the total dynamic memory usage of a network is an order of magnitude more than the static memory as images are batched and passed through the network in sets of 50 to 250 for better accuracy (stochastic gradient descent with mini-batching). This increases the memory usage by approximately the factor of the mini-batch size. Additionally, gradient matrices need to be stored for every layer for back-propagation and this again increases the memory, this time by a factor of about 2x.

In a CNN the total amount of static memory is the size of each filter times the number of filters in a layer. The filter weights are added to the weights in the classification layer (MLP) to make up the total static memory. **In a CNN more than 90% of the total static memory is in classification layers.** The total amount of dynamic memory is equal to the size of the resultant feature map after applying every filter through the input feature maps. This is added to the amount of dynamic memory of the activations of the fully connected layer, which is negligible compared to the memory needed for the resultant feature maps. **In a CNN more than 90% of the total dynamic memory is in feature maps.**

To explore this calculation further Figure 10 summarizes the parameters and memory usage of VGGNet[1], a deep 16-layer CNN and winner of ImageNet competition in 2014. Note that the number of parameters in the convolution layers are significantly less than the parameters in the fully connected layers. This is because **the kernel weights themselves are small shared resources local to the entire image.** In contrast, the number of parameters in the last three fully connected layers is an order of magnitude more. While this is a rough approximation and does not consider the diverse set of software optimizations that have been implemented, we see that compared to actual memory requirements for VGGNet-256 reported by Nvidia[7] in Figure 7 this approximation is acceptable (30GB calculated vs. 28GB actual).

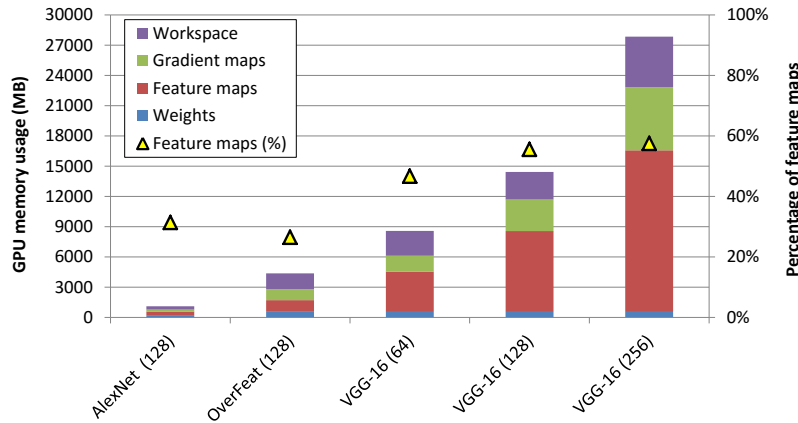


Figure 7: Experimental results from Nvidia about the memory usage and breakdown for various CNNs. Most of the memory usage is in the feature maps[7].

5.2 Neural Network Computational Requirements

The naive way to perform convolutions involves a series of nested for loops which iterate through height, width, and depth for the input feature map and the filters. This method takes $(n - k + 1)^2 k^2$ operations and is not scalable to very deep networks. The preferred method is to decompose these 4D image volumes, or tensors, in a way that we can leverage parallel compute kernels. In modern libraries, the operations performed in a CNN are matrix-matrix multiplies and vector-matrix multiplies based on the image to column decomposition[8]. For an image of size $H \times W \times C$ a single receptive field will have size $K \times K \times C$. This receptive field will be reshaped to a column of $K^2 \times C$ elements

and a matrix of $K^2 \times C \times N$ where N is the number of receptive fields in the input feature map. We can perform a similar operation for the filters, reshaping the filters of size $K \times K \times C$ to rows and create a matrix of $K^2 \times C \times D$ where D is the number of filters being used. At this point, we perform a matrix multiply to obtain a $D \times N$ result which we can reshape to produce an output tensor. In terms of complexity this method which was implemented in an early version of Caffe[9] has a time complexity of $\mathcal{O}(N^2 K^2)$ for the im2col operation and an additional $\mathcal{O}(N^2 K^2 F)$ for the dense matrix multiplies. This operation is summarized in Figure 9.

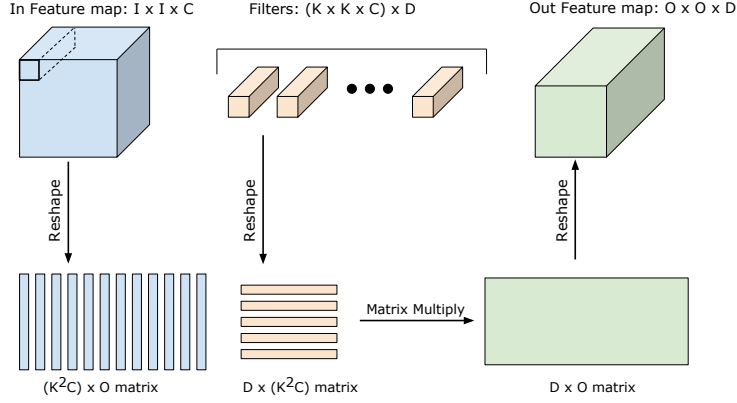


Figure 8: The im2col approach of convolution. The feature maps and filters are decomposed into matrices and a matrix multiply is performed to obtain the new output feature map.

Another area of active research involves leveraging the convolution theorem and decomposing the operation into a set of FFTs. Instead of using direct computation, an FFT is performed on the filters and feature maps and combined through matrix multiply operations. To produce the output feature map, an inverse FFT then needs to be computed. The complexity of the FFT based method requires $\mathcal{O}(n^2 \log n)$ for each FFT and the pointwise products require $4n^2$ calculations for a total of $6Cn^2 \log n + 4n^2$ operations[10] where C is a constant. While this method can significantly reduce the amount of computation for convolutions it incurs a large memory overhead when the sizes of the kernels do not match the size of the feature map as the inputs need to be padded[11]. This method works the best during later stages of a CNN after the inputs have been downsampled several times and the size of the feature map matches the filter size. There has been interest in other

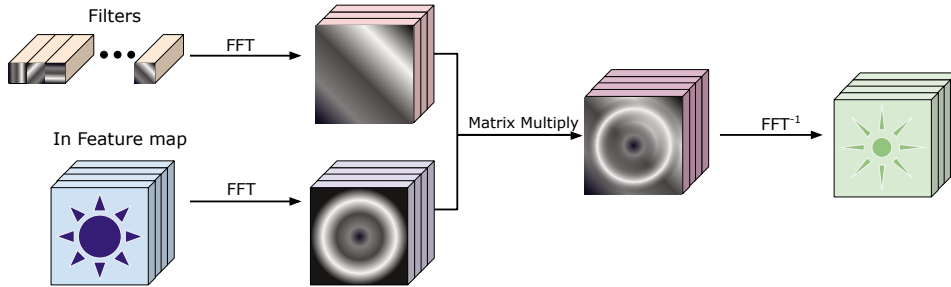


Figure 9: Taking an FFT approach to convolution. There is a high memory overhead associated with this especially if the size of the filters do not match the size of the input feature map. This is the case during the early layers of a CNN.

methods for convolution that involve reducing the computation time of matrix multiplication given a specific kernel size (e.g. 3×3 filters). Using Strassen's algorithm one can reduce the complexity of a matrix-matrix multiply from the naive $\mathcal{O}(N^3)$ to $\mathcal{O}(N^{2.81})$ by reducing the amount of multiplies.

This technique was implemented for 2x2 and 3x3 kernel sizes and shows massive speedup on VGG for small batch sizes[12].

6 The Communication Bottleneck

There are two main approaches to parallelize a CPU/GPU cluster for neural computation. The first is to use GPUs in a **data parallel** mode where each compute unit has the entire model in memory and computes gradients on different parts of the training data. The second is to employ **model parallelism** to spread the model parameters over all of the compute units, such as GPUs, and synchronize the parameters and gradients after each layer. These methods are employed when the size of the network itself or the size of each mini-batch does not fit in a compute unit. For both of these approaches, the cost of communication and bandwidth begin to limit scaling benefits when training very deep networks on large clusters resulting in marginal speedup and low accuracy[13].

6.1 Data Parallelism

In the data parallel approach the model is stored inside each GPU and a different mini-batch of the training data is fed through the GPUs. The forward pass is computed independently for each GPU and for the backward pass the entire gradient matrix need to be synchronized (averaged) and distributed across all GPUs. The pitfalls of using this method are clear when considering two weight matrices (3 fully connected layers) of size 4096x512 as in the Alexnet[6] architecture. With a 6GB/s PCIe link and 2 TFLOP GPUs it takes $2.09\mu s$ to compute the gradient for each sample and $2.73\mu s$ for each GPU to receive 4096 floats. In a multi-node GPU cluster, this approach would involve having to aggregate gradients in each node and transferring large matrices across a network. *This approach makes the most sense when the compute costs per parameter are high and total parameters are low*[14].

6.2 Model Parallelism

Alternatively, a model can be paralleled by distributing the parameters of a layer on multiple GPUs. In contrast to the data parallel technique, each GPU is fed the same mini-batch at the same time. This approach requires frequent communication to synchronize and aggregate all the parameters in each layer and can work well on a single GPU cluster with a shared high speed communication bus. Problems arise when scaling up to several clusters the cost of synchronization and aggregation across machines at this rate can quickly overwhelm the communication costs. This approach is optimal for networks with many parameters (fully connected layers) as we can split the model across the GPUs and only need to communicate small dense matrices.

The COTS HPC system [15] successfully uses this model parallel approach in a distributed GPU cluster with a highly tuned CUDA kernel, custom MPI interface, and Infiniband interconnects. This technique obtains just under linear speedup (80% of maximum) of computation with a 64 GPU cluster running an 11 billion parameter sparse autoencoder. One of the insights that allowed them to obtain this speedup was the observation that this network had many sparse weight matrices. The authors noted that in generic sparse linear algebra code tends to be much slower than the highly optimized libraries for dense matrices and opted to decompose the computation of $Y = WX$ as a large number of small dense matrix multiplies. This allowed them to saturate compute cores and utilize 800GFLOPS on a 1FLOP peak machine in contrast to 300 GFLOPS without this optimization. It is important to note that the sparse autoencoder used to benchmark this system is a fully connected neural network for unsupervised learning and has a different network architecture with respect to CNNs.

6.3 Hybrid Approaches

The idea of splitting neural networks using either model and data parallelism has been explored in older systems [16, 17] but were primarily tested on multi-layer perceptrons. With CNNs both types of parallelism can be utilized with the memory intensive convolution layers running on data parallel mode and the dense fully connected layers in model parallel mode. This distinction between approaches can also be made in terms of neuron activity - we utilize model parallelism for layers where the neuron activity is high (fully connected) and data parallelism for layers where the compute per weight is high (convolution layers). This technique, popularized by [14] shows that utilizing

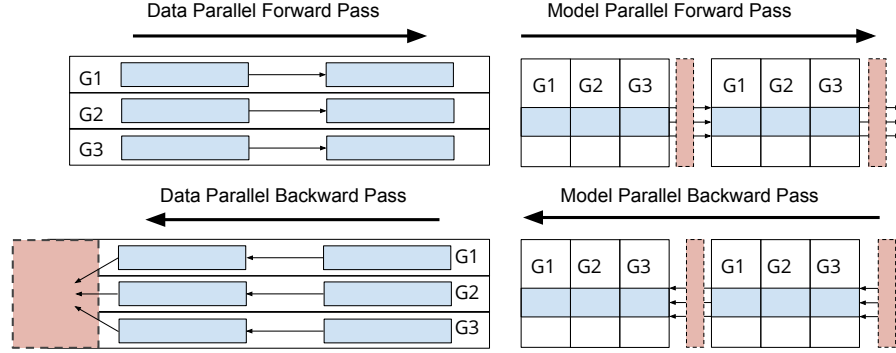


Figure 10: Difference between the forward and backward pass for data and model parallelism. In the data parallel approach GPUs G1, G2, and G3 work independently and total parameters in the end. In model parallel, a small amount of communication is done every layer to synchronous activations.

hybrid parallelism allow a cluster of 8 GPUs to utilize up to 80% of the total theoretical bandwidth. The only issue is that there is a slight drop in accuracy but this can be mitigated with larger datasets.

7 Taxonomy and Evaluation of Neural Systems

Hardware support for neural network falls primarily into three categories. The first category is **Technology Enabled Acceleration (TEA)**. This category involves new or emerging technology that has shown promising results in neural net based algorithms. The family of DianNao papers[18, 19] works primarily on the problem of the inference by designing accelerators for CNNs ranging from low power camera ASICs[20] to high throughput supercomputer accelerators[21]. Other recent work has explored using memristor based technologies for the repeated store/compute operation in neural networks[22, 23, 24]. In one example memristor crossbar arrays are utilized to store input weights and perform the dot-product in an analog manner[25]. While this has been explored before the ISAAC accelerator is one of the first that develops an entire crossbar based accelerator which performs 14x better than the state-of-the-art Dadianao supercomputer accelerator. It is important to note that almost all TEA solutions focus exclusively on improving inference time computation.

Technology Enabled Acceleration		
DianNao	Inference	MLP accelerator for embedded devices
DaDianNao	Inference	High performance tile based ASIC
ISSAC	Inference	CNN ASIC with memristor crossbar arrays
System Enabled Acceleration		
DistBelief/TensorFlow	Train	CPU cluster with distributed learning algorithm
Project Adam	Train	CPU cluster with hybrid parallelism for CNNs
COTS HPC	Train	GPU cluster using off the shelf commodity parts
Algorithm Enabled Acceleration		
BinaryConnect	Train	Binarize activations
Batch Normalization	Train	Improve speed and accuracy
Gradient-less Methods	Train	Apply ADMM to MLPs
SqueezeNet	Both	Convert CONV layers to compressed layers

Table 1: Examples from each category of the neural system taxonomy

The second category is **System Enabled Acceleration (SEA)**. SEA innovations focus on how to scale large scale GPU/CPU clusters for neural networks. For the most part these system level improvements are mostly at the software level with optimizations to minimize communication costs. One of the first modern era systems papers focused on this problem was the Dist Belief system from Google[13]. Although their benchmark was a sparse autoencoder which is very different from a

CNN, they showed that the way the net was distributed across the cluster effects evaluation accuracy. To overcome this limitation they proposed new distributed training algorithms that scaled better but ultimately this scaling broke down at around 2000 CPU cores at which point accuracy began to drop. Shortly after, Stanford developed the COTS HPC system, a distributed GPU cluster with off the shelf commodity parts. Their system utilized model parallelism and transformed sparse operations to dense to maximize the GPU throughput. Using these techniques they obtained better scaling at less training time compared Google's Dist Belief CPU cluster[15]. Microsoft's distributed system, Project Adam [26] develops a cluster for CNNs and partitions the network across the layer instead of across the network like DistBelief. Additionally, Project Adam make a better case for how to load balance between the worker machines and the parameter server and places more computation on the parameter server to lower the communication bottleneck. All SEA innovations are focused exclusively on enabling deeper nets, speeding up training time, and improving training accuracy.

The final category is **Algorithm Enabled Acceleration (AEA)**. This category focuses exclusively on innovations in neural network theory to improve training and inference time computation. There has been a large amount of effort in leveraging the error resiliency of neural networks for limited numerical precision computation[27]. While this phenomena has been observed before [28, 29], more recently analysis on deep networks show that with only a few weights per feature it is possible to predict more than 95% of the weights in a network without loss of accuracy[30]. This has led to a branch of research looking to compress or hash networks to reduce the number of weights and memory footprint[31, 32].

Further down the theoretical side, some work has been done to create new algorithms specifically for training that both improve accuracy and scale well to large distributed systems. One technique that has begun to show promise involves moving away from gradient based methods and toward other nonlinear optimization techniques such as the alternating direction method of multipliers(ADMM)[33]. [34] show that this method can scales well to large cluster sizes when tested on a 5000+ core cluster running a three layer MLP.

In the next section we explore each of these three areas with specific examples.

Technology Enabled Acceleration

The first paper in a series of neural network accelerators to show a synthesized (place & route) accelerator hardware was DianNao[18]. The DianNao accelerator which focuses primarily on fully connected layers has a pipelined architecture with three pipeline stages for the essential computations performed in a MLP - multiply, accumulate, apply transfer function. Data is passed from an off-chip DRAM into on chip scratchpad memory to store intermediate buffers of input and output neurons. These buffers can only process a small amount of neurons at a time. By placing data closer to the logic some of the off-chip access are reduced but the main problem with this implementation is the bandwidth required for the rest of the off-chip accesses to obtain the next set of inputs. It requires a link operating at at 470GB/s which is unrealistically high considering the best memory controller on the market primarily can only provide between 200-250GB/s. In addition, the authors make a claim that this accelerator can be applied to CNNs but this implementation actually scales very poorly to CNNs as DianNao would decompose images into 1D vectors before compute convolutions rather than exploiting locality in the the 2D feature maps.

ShiDianNao[20] tries to address the issue of poor convolution implementation by designing an architecture specifically for the high amount of matrix-matrix multiplies in the convolution layers. This system was built as a peripheral co-processor in camera devices and would sit near the sensor and process data immediately after the image is taken. In this architecture an on-chip SRAM is used to store all of the weights and neurons and a 2D mesh of elements (PEs) exploits the regular structure of CNN matrix based computation. Similar to DianNao the on-chip memory, SRAM in this case, stores the input and output neurons and have to store the feature maps and activations as well. Paradoxically this means this architecture can only store 64k of parameters which is three orders of magnitude less parameters than AlexNet. This is without even beginning to consider the memory needed to store the output feature maps which is several times more. The next iteration of DianNao, DaDianNao[21] broadens the scope of this system by taking a tile based approach for they call a neural "supercomputer". The architecture uses a tile based architecture to compensate for the high off chip DRAM accesses in the original DianNao architecture and make two main optimizations to overcome this limitation. The first is to utilize high storage density eDRAM banks to store weights

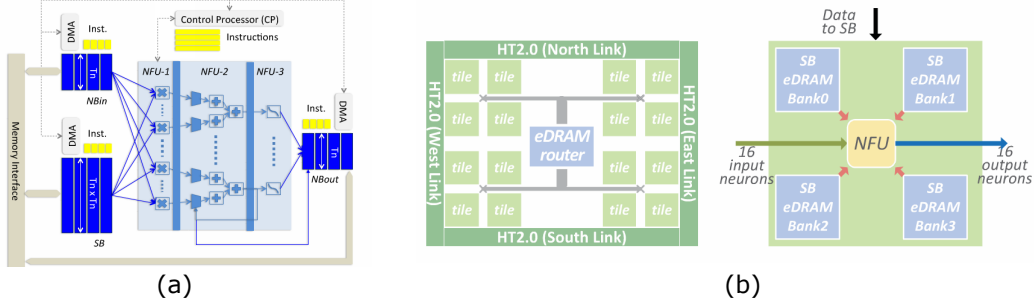


Figure 11: (a) DianNao accelerator. (b) The right side shows a single tile with an NFU (neural functional unit) which is similar to (a). The left shows a chip with 16 tiles.

and the second was to adopt a multi-node system to distribute weights across a cluster. One tile can be thought of as a single DianNao accelerator with four high storage density eDRAM banks to store weights with nodes. Each node is networked internally through a fat tree which broadcasts neurons to each tile. The authors envision this system as a datacenter level chip for which many nodes can be interconnected and report up to a significant speedup over state of the art GPUs. One issue with this system is that the size of the internal and external eDRAM are far too small to accommodate even a moderately sized image recognition network such as AlexNet. Although the claim is that this network works for CNN training as well as inference, a 2GB network like Alexnet equates to a 64-node cluster with 32MB of eDRAM per node. At 64 nodes the energy cost of the interconnects between nodes starts to dominate energy with 30% of total energy going to communication costs, most of which is in the fully connected layers.

As noted before, one of the main problems with the DaDianNao is the energy cost of intra-node communication between the fully connected layers. The convolution layers can stay largely localized to a node as each filter can be in a different node but the dense classification layers are the source of the communication overhead. The EIE: Efficient Inference Engine[35] accelerator attacks this problem by focusing exclusively on improving the throughput on the fully connected layers on the network. EIE is a good example of integrating AEA optimizations into a TEA technique. Deep Compression [36] is a technique to reduce the number of parameters through compression and encoding but the weight index need to be looked up during inference. Additionally, the increased sparsity of the network also requires systems for efficient sparse matrix operations. EIE approaches this problem by encoding the sparse weights in a denser column format and operates on this new data structure entirely in SRAM. Compared to the DaDianNao accelerator EIE is able to perform at 2.9x, 19x and 3x better throughput, energy efficiency and area efficiency. More about deep compression is discussed in the AEA section later in the paper.

System Enabled Acceleration

The training algorithm for neural networks is a computationally intense procedure due to the nature of the back-propagation algorithm discussed in Section 3. There are three main steps when performing training - the forward pass, backward pass, and the gradient update. The large memory and computational footprint of training come from the fact that the images are trained in mini-batches and have to perform several million updates in an iterative manner using stochastic gradient descent optimization. Mini-batch processing quickly blows up the memory footprint of the network and the serial formulation of SGD makes it less conducive to parallelization.

The DistBelief[13] system from Google first tackled this problem by throwing away the notion that updates in SGD had to be synchronous and exploited model parallelism within a machine and data parallelism with several model instances or replicas. Model replicas refer to a single multi-machine compute node that distribute the network model across machines. During training, each model replica is fed a different mini-batch of data and sends the gradients to a central parameter server. The parameter server updates the model parameters with the gradient values and then distributes the new gradients among the model replicas. Each model replica is stored in a 4 machine node exploiting the idea of model parallelism reducing the per machine load. To perform the actual training the DistBelief system tested two separate approaches to the training problem. The first approach was to

Table 2: Comparison of various TEA based accelerators. The Eyeriss, ISSAC and EIE accelerators were omitted due to the vast number of configurations which made it difficult to obtain a fair comparison

publication	type	platform	theor. GOp/s	power W	power eff. GOp/s/W
Cavigelli et al. [37]	CPU	Xeon E5-1620v2	118	230	0.51
Cavigelli et al. [37]	GPU	GTX780	3977	sd:200	20
cuDNN R3 [38]	GPU	Titan X	6600	d:250	26
Cavigelli et al. [37]	SoC	Tegra K1	365	s:11	33
Microsoft [39]	FPGA	Arria 10 GX1150	1400	40	35
ConvEngine [40]	synth.	45nm	410	c:1.0	410
DianNao [18]	layout	TSMC 65nm	452	c:.485	0.93
DaDianNao [21]	layout	ST 29nm LP	2090	n:15.9	130
ShiDianNao [20]	layout	TSMC 65nm	128	c:0.32	400
NeuFlow [41]	layout	IBM 45nm SOI	1280	d:5	256
NeuFlow [41]	layout	IBM 45nm SOI	320	c:0.6	533
HWCE [42]	layout	ST 28nm FDSOI	37	c:0.18	205
HWCE [42]	layout	ST 28nm FDSOI	1	c:0.73m	
Origami [43]	silicon	umc 65nm	196	c:0.51	384
Origami [43]	silicon	umc 65nm	74	c:0.093	795

have a central parameter server with model replicas asynchronously updating the parameter server with a variant of asynchronous SGD, Downpour. With Downpour SGD the model replicas are run independently of each other and each shard in the parameter server updates independently. Before processing a mini-batch each machine in the model replica requires a different piece of the model parameters to make up the full network. After processing, the corresponding gradients are sent to the parameter server.

The second technique uses a quasi-Newton optimization method known as L-BFGS, with the asynchronous variant dubbed SandBlaster. The main idea in the SandBlaster based architecture is to have distributed parameter manipulation. Instead of the direct communication between the nodes and the parameter server an auxiliary machine issues commands to coordinate the optimization process by issuing commands for multiply, dot product, and scaling. Unlike Downpour which has access to multiple data shards per node, Sandblaster has one centralized data block. The reason for this approach is because the BFGS algorithm requires computation of the gradient on the whole dataset to make an update. While this can mean a better optima it also means it does not scale well with the number of examples as the computational cost per gradient iteration is high[44]. The authors found that for both of these implementations, the asynchronous techniques were able to outperform their synchronous variants on the distributed system.

When looking at how these algorithms scaled both of these methods started to show diminishing returns after 5000 cores. In fact, Downpour SGD began to lose accuracy training beyond 1500 cores. While Sandblaster did not lose accuracy as the system scaled up, there was almost no improvement in training time going from 2000 machines to 5500 machines. One reason for this could be that the cluster was not saturated enough to take advantage of the number of cores, but the network was trained on a very large 1.7 billion parameter neural network with 21 million images from ImageNet.

Microsoft’s work on Project Adam[26] recognizes that communication between model replicas and the parameter server was the primary bottleneck when scaling up the system. Adam adapts an asynchronous SGD algorithm similar to DistBelief, but does not use the same data parallelism scheme with multiple model replicas updating the parameter server. Rather, they chosen to split the model on multiple machines across layers, i.e. every machine will hold one vertical slice of the whole model. Using this approach they are able to make two further optimizations given that every machine has a very small piece of the entire network. Now the working set of the network can completely fit into the L3 cache of a machine. To attack the problem of high communication between the model replica and parameter server Adam chooses to off-load some of the computation of the gradient updates into the parameter server. In the DistBelief system, the purpose of the parameter server was just to take the gradient updates that each machine calculated and apply it to the parameter server. This can lead to a

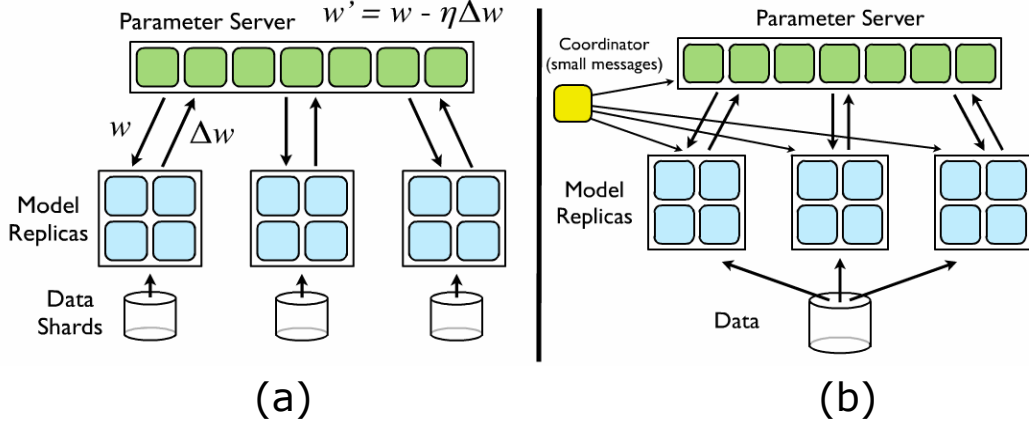


Figure 12: (a)-Downpour: The parameter server exchanging weights with the workers which pass new weights up to the parameter server. (b)-SandBlaster: A coordinator manages communication and one entire batch is sent to all model replicas

bottleneck in the network as the gradient maps take a large pool of memory as they are aggregated over each layer. Instead Adam proposes to just send the error gradients calculated after every layer to the parameter server and let the parameter server calculate the gradient for every machine and apply it. This has the benefit of load balancing the computation between model and parameter machines and overcomes the memory bandwidth problem that would occur when increasing the number of active cores per machine. Similar to the DistBelief paper Adam shows that having asynchronous updates to gradient descent acts as a form of regularization and improves overall accuracy on the test set. Compared to DistBelief this system was tested on a quarter of the cluster size so it is difficult to speculate how it scales beyond 1000 cores. In DistBelief scaling was also linear until about 2000 cores at which point the scaling tapered off and accuracy began to drop[13].

Algorithm Enabled Acceleration

For years the development of new neural networks and better training algorithms has been largely orthogonal to what kind of hardware might drive the usage of these algorithms. As the application scope and potential of neural networks continues to expand some research has been placed in developing new techniques at the algorithm level to enable low power inference or scalable parallel computation. In the realm of inference, finding a way to lower the memory and energy footprint of neural networks is something that has traditionally been approached from a purely hardware standpoint as seen by examples such as the DianNao[18] and ORIGAMI[43]. To hardware researchers the algorithms are as a recipe that will stay essentially the same and hardware just has to optimize the tools. In reality, the state of the art in neural network research is changing so quickly any chip that is built can be rendered obsolete in a matter of months. Neural networks are nowhere close to being a mature computation and any chip built either has to target the most basic neural network primitives such as matrix multiples or convolution[40] which only optimize throughput but not necessarily memory. To place these powerful algorithms in low power devices such as mobile phones, contact lenses, and next generation IoT devices an approach that both optimizes throughput, energy, and memory must be taken. One way to do this is to bring in some insights and ideas from the theoretical side.

It has long been noted that neural networks are resilient to perturbations and these small variances even have the potential to enhance generalization in neural networks[30]. Recent papers have explored reducing parameter size and resolution during both inference and training by compressing or approximating weights. By finding a way to relax the constraints of having high precision operators or reducing the number of multipliers, both energy and storage space can be reduced. While most of the work in neural compression is applied toward inference, as training requires the network to accumulate small changes in the parameters with high precision, there has been some work exploring the space of limited precision training parameters[27]. One way to approach this problem is to reduce the amount of multiplies by having low precision multiplies with high precision accumulates. In

this approach, the parameters for updates are kept at high precision while the number of bits for forward and backward propagation are varied. The results showed that by utilizing 12-bit dynamic fixed-point computation they are able to maintain accuracy compared to 32-bit computations[45]. These experiments open the possibility that the weights in a network can be low precision as long as the parameter updates are high precision. This was the approach taken for the BinaryConnect series of papers that take this approach to an extreme by binarizing the weights during the forward and backward pass. Psuedocode for this operation is shown in Figure 13. BinaryConnect was shown to

Algorithm 1 SGD training with BinaryConnect. C is the cost function for minibatch and the functions $\text{binarize}(w)$ and $\text{clip}(w)$ specify how to binarize and clip weights. L is the number of layers.

Require: a minibatch of (inputs, targets), previous parameters w_{t-1} (weights) and b_{t-1} (biases), and learning rate η .

Ensure: updated parameters w_t and b_t .

1. Forward propagation:

$w_b \leftarrow \text{binarize}(w_{t-1})$

For $k = 1$ to L , compute a_k knowing a_{k-1} , w_b and b_{t-1}

2. Backward propagation:

Initialize output layer's activations gradient $\frac{\partial C}{\partial a_L}$

For $k = L$ to 2, compute $\frac{\partial C}{\partial a_{k-1}}$ knowing $\frac{\partial C}{\partial a_k}$ and w_b

3. Parameter update:

Compute $\frac{\partial C}{\partial w_b}$ and $\frac{\partial C}{\partial b_{t-1}}$ knowing $\frac{\partial C}{\partial a_k}$ and a_{k-1}

$w_t \leftarrow \text{clip}(w_{t-1} - \eta \frac{\partial C}{\partial w_b})$

$b_t \leftarrow b_{t-1} - \eta \frac{\partial C}{\partial b_{t-1}}$

Figure 13: Psuedocode for BinaryConnect[46]. The binarization process happens in the forward propagation but not in the parameter updates.

have close to state-of-the-art results while reducing the number of multiplies by 2/3. The authors note that by binarizing the network in this way it acts as a form of regularization which improves the overall network performance[46].

Other methods of compressing such as hashing the weights during training[31] has also been explored. In this method, parameters are hashed in a way that connections within the same hash bucket share a single parameter value. At this point a virtual weight matrix is created which turns the fully connected network into a sparsely connected network with shared weight filters, similar to the kernels in a CNN. The results show that this technique works well when compressing the network to 1/8 its normal size and going to 1/64 of the size error does increase slightly relative to an uncompressed network (1.5% to 3%). HashNets were tested on a small benchmark of 3 and 5 layer MLPs but the accuracy begins to drop as the number of network layers increases. This may become an issue occurs when trying to applying this to deep unsupervised fully connected networks such as an autoencoder for which hashed training may severely hurt test accuracy. Another interesting experiment the authors performed was to expand the hashed virtual network larger than the original size. While this obviously does not save any space they noted that expanding the virtual weight matrix by 4-8x, improved accuracy. This property may be attributed to the fact that network now has a more rich space to learn features from with the increased number of weights.

Applying compression to the inference process has a wide range of implications for embedded systems and has spawned a series of work in compressing the weights of a neural network[47, 36]. The main idea here is to take an already trained network and prune all connections with weights below a certain threshold. Then the network is retrained on this now sparse network and the process is repeated until acceptable accuracy is achieved. The results of this technique show a small increases the error compared to the reference network, VGGNet, while compressing the network at a rate of 9-13x. It should be noted that this is not an optimization to lower the memory usage during training and in fact increases total training time as there are now two training phases. The authors address this concern by stating that the cost of training the network a second time only happens when the model is ready for deployment rather than during model prototyping but this may be an issue in the future with as networks become deeper. Modern state of the art networks can take months to train and the iterative pruning and training processes incurs an additional overhead of about 2.5x. A later work,

Deep Compression[36] further improves on this idea by adding two more stages to the compression pipeline, quantization and Huffman encoding. The idea behind quantization is similar to [48] in which the authors cluster weights using k-means in order to use less bits per weight. In that paper quantization was able to reduce network parameters by about 16x without loss in top-five accuracy. In Deep Compression, the largest benefit comes from the pruning-retraining phase with about 10x improvement, the quantization reduces the remaining network by 3x and the encoding reduces the remaining weights to about 1.5x. This results in a 35-49x reduction on the original network with essentially no loss in accuracy benchmarked against AlexNet.

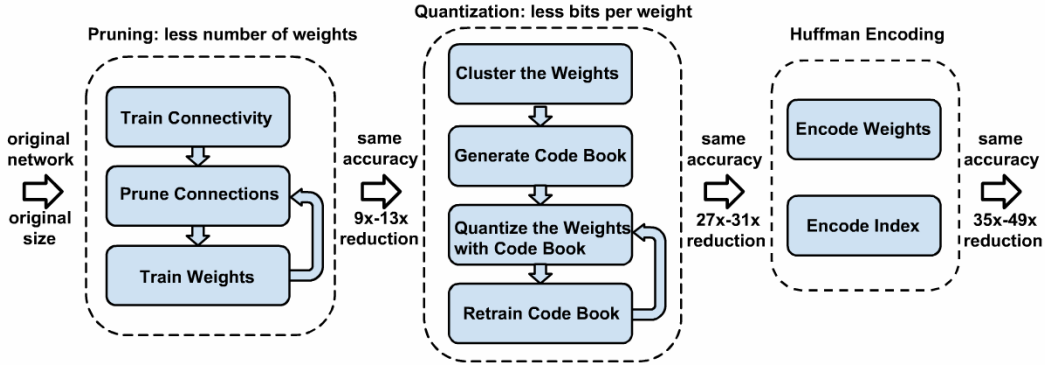


Figure 14: Deep Compression pipeline

Reducing training time and improving parallelization with new optimization techniques is an area where less effort has been focused due to the complexity of the training problem and generally marginal improvements in model accuracy. One way to approach this problem is to attack the communication bottleneck by reducing parameters and lower the amount of data sent across servers. Where Deep Compression focused exclusively on compression inference, the SqueezeNet architecture[32] introduces a new approach for developing CNNs with 50x fewer parameters. In this architecture, a normal convolution step is replaced with squeeze-expand stages which first reduces the total parameters with 1x1 filters and then expands them to stacked 1x1 and 3x3 filters. While parameter reduction is useful, the memory requirements go up drastically. The AlexNet baseline that they compare against has 240MB of parameters and uses about 2GB of memory for training with batch size of 128. The authors do not report what the batch size of the network during training is, but the default batch-size in the released code is 512. Performing the same back of the envelope calculation on SqueezeNet as performed on VGG, SqueezeNet uses 30GB of memory an increase of 15x over AlexNet. Note that 30GB is the same memory footprint of VGG during training but where VGG had an error rate of 6.8% SqueezeNet has an error rate of 20%. In summary SqueezeNet is able to reduce parameter of an equivalent network by 50x but increases memory usage by 2-10x and results in 3x less accuracy than a network with the same memory footprint.

Moving away from compression, a few techniques incorporated directly into the the training process have shown to both improve accuracy and reduce total training steps using techniques like batch normalization[49]. As a pre-processing steps in neural networks data is normalized to speedup training and reduce chances of being stuck in a local optima. As the inputs are processed through a neural network the benefits of normalization loses its effectiveness as the parameters change to become larger or smaller. This phenomena is refereed to as internal covariate shift and the authors propose normalising data in each mini-batch in the middle of the network. This reduces the total number of training steps by 14x, as the network can tolerate more noise and learning rates can be raised, and improves accuracy over the 22-layer GoogLeNet baseline without having to increase the size of the network.

8 Conclusion

This survey reviews state of the art algorithms in neural networks and the various approaches taken to optimize speed, accuracy, and energy. The hardware, systems, and theory communities are all

working toward one goal to enable faster and more accurate networks and in general each community tackles a different neural problem - hardware community works on inference, systems works on training, and theory works on both. While this has worked well in the past, new research ideas in this area cannot be developed without a good understanding in all three. The most interesting papers are the ones that can synthesize ideas from two or more of these groups to produce a system well informed theoretically and grounded in real hardware. When AlexNet won the ILSVRC in 2012, it was the result of a new network architecture trained with an efficient software kernel and run on high performance hardware. It was an understanding and optimization of all three areas TEA-SEA-MEA that enabled AlexNet to outperform decades of computer vision research and change the landscape of every research community.

References

- [1] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [2] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” 2014.
- [3] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” 2015.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers: Surpassing Human-Level performance on ImageNet classification,” 2015.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015.
- [6] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in neural information processing systems*, pp. 1097–1105, 2012.
- [7] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, “Virtualizing deep neural networks for Memory-Efficient neural network design,” 2016.
- [8] Y. Jia, “Learning semantic image representations at a large scale,” 2014.
- [9] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, “Caffe: Convolutional architecture for fast feature embedding,” in *Proceedings of the ACM International Conference on Multimedia*, pp. 675–678, ACM, 2014.
- [10] M. Mathieu, M. Henaff, and L. Yann, “Fast training of convolutional networks through FFTs,” 2013.
- [11] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, “cuDNN: efficient primitives for deep learning,” 2014.
- [12] A. Lavin and S. Gray, “Fast algorithms for convolutional neural networks,” 2015.
- [13] J. Dean, G. Corrado, R. Monga, and K. Chen, “Large scale distributed deep networks,” 2012.
- [14] A. Krizhevsky, “One weird trick for parallelizing convolutional neural networks,” 2014.
- [15] A. Coates, B. Huval, T. Wang, D. Wu, B. Catanzaro, and N. Andrew, “Deep learning with cots hpc systems,” in *Proceedings of the 30th international conference on machine learning*, pp. 1337–1345, 2013.
- [16] P. Farber and K. Asanovic, “Parallel neural network training on Multi-Spert,” 1997.
- [17] G. Dahl, M. A. and T. Newhall, “Parallelizing neural network training for cluster systems,” 2008.
- [18] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, “Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning,” in *ACM Sigplan Notices*, vol. 49, pp. 269–284, ACM, 2014.
- [19] D. Liu, T. Chen, S. Liu, J. Zhou, S. Zhou, O. Teman, X. Feng, X. Zhou, and Y. Chen, “Pudiannao: A polyvalent machine learning accelerator,” in *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 369–381, ACM, 2015.
- [20] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, “Shidiannao: shifting vision processing closer to the sensor,” in *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pp. 92–104, ACM, 2015.
- [21] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, *et al.*, “Dadiannao: A machine-learning supercomputer,” in *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 609–622, IEEE Computer Society, 2014.
- [22] M. Bojnordi and E. Ipek, “Memristive boltzmann machine: A hardware accelerator for combinatorial optimization and deep learning,” 2016.
- [23] S. Hamdioui, L. Xie, H. Nguyen, and M. Taouil, “Memristor based computation-in-memory architecture for data-intensive applications,” 2015.
- [24] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, “Prime: A novel processing-in-memory architecture for neural network computation in rram-based main memory,”

- [25] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars,"
- [26] T. Chilimbi, Y. Suzue, J. Apacible, and K. Kalyanaraman, "Project adam: Building an efficient and scalable deep learning training system," 2014.
- [27] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," 2015.
- [28] M. Hoeftfeld and S. E. Fahlman, "Learning with limited numerical precision using the cascade-correlation algorithm," *Neural Networks, IEEE Transactions on*, vol. 3, no. 4, pp. 602–611, 1992.
- [29] J. L. Holt and T. E. Baker, "Back propagation simulations using limited precision calculations," in *Neural Networks, 1991., IJCNN-91-Seattle International Joint Conference on*, vol. 2, pp. 121–126, IEEE, 1991.
- [30] M. Denil, B. Shakibi, L. Dinh, M. Ranzato, and N. de Freitas, "Predicting parameters in deep learning," 2013.
- [31] W. Chen, J. T. Wilson, S. Tyree, K. Q. Weinberger, and Y. Chen, "Compressing neural networks with the hashing trick," *arXiv preprint arXiv:1504.04788*, 2015.
- [32] F. N. Iandola, M. W. Moskewicz, K. Ashraf, S. Han, W. J. Dally, and K. Keutzer, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and < 1mb model size," *arXiv preprint arXiv:1602.07360*, 2016.
- [33] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends® in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [34] G. Taylor, R. Burmeister, Z. Xu, B. Singh, A. Patel, and T. Goldstein, "Training neural networks without gradients: A scalable ADMM approach," 2016.
- [35] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: Efficient inference engine on compressed deep neural network," *arXiv preprint arXiv:1602.01528*, 2016.
- [36] S. Han, H. Mao, and W. Dally, "A deep neural network compression pipeline: Pruning, quantization, huffman encoding," 2015.
- [37] L. Cavigelli, M. Magno, and L. Benini, "Accelerating real-time embedded scene labeling with convolutional networks," in *Proceedings of the 52Nd Annual Design Automation Conference, DAC '15*, (New York, NY, USA), pp. 108:1–108:6, ACM, 2015.
- [38] S. Chintala, "convnet-benchmarks," 2015.
- [39] K. Ovtcharov, O. Ruwase, J.-Y. Kim, J. Fowers, K. Strauss, and E. S. Chung, "Accelerating deep convolutional neural networks using specialized hardware," *Microsoft Research Whitepaper*, vol. 2, 2015.
- [40] W. Qadeer, R. Hameed, O. Shacham, P. Venkatesan, C. Kozyrakis, and M. A. Horowitz, "Convolution engine: balancing efficiency & flexibility in specialized computing," in *ACM SIGARCH Computer Architecture News*, vol. 41, pp. 24–35, ACM, 2013.
- [41] C. Farabet, B. Martini, B. Corda, P. Akselrod, E. Culurciello, and Y. LeCun, "Neuflow: A run-time reconfigurable dataflow processor for vision," in *Computer Vision and Pattern Recognition Workshops (CVPRW), 2011 IEEE Computer Society Conference on*, pp. 109–116, IEEE, 2011.
- [42] F. Conti and L. Benini, "A ultra-low-energy convolution engine for fast brain-inspired vision in multicore clusters," in *Proceedings of the 2015 Design, Automation & Test in Europe Conference & Exhibition*, pp. 683–688, EDA Consortium, 2015.
- [43] C. Accelerator, "Origami: A 803 GOP/s/W convolutional network accelerator," 0.
- [44] J. Ngiam, A. Coates, A. Lahiri, B. Prochnow, Q. V. Le, and A. Y. Ng, "On optimization methods for deep learning," in *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, pp. 265–272, 2011.
- [45] M. Courbariaux, Y. Bengio, and J. David, "Training neural networks with low precision multiplications," *arXiv preprint arXiv:1412.7024*, 2014.

- [46] M. Courbariaux, Y. Bengio, and J.-P. David, “Binaryconnect: Training deep neural networks with binary weights during propagations,” in *Advances in Neural Information Processing Systems*, pp. 3105–3113, 2015.
- [47] S. Han, J. Pool, J. Tran, and W. Dally, “Learning both weights and connections for efficient neural network,” 2015.
- [48] Y. Gong, L. Liu, M. Yang, and L. Bourdev, “Compressing deep convolutional networks using vector quantization,” *arXiv preprint arXiv:1412.6115*, 2014.
- [49] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” 2015.